E-INFO • FACAPE • TURNO DA TARDE (14H00–18H00)

Redes.tf // Parte 2: Cloud NAT, VPC Peering e Módulos

Tabelas de Roteamento, Saída Segura com Cloud Router e Cloud NAT, Interconexão Hub-and-Spoke via VPC Peering, Desafio do Mural Ao Vivo e Terraform Modules.

Facilitador: Luan Sousa — Cloud Engineer @ SantoDigital | Google Cloud Digital Leader | CC FACAPE

Por que o ping 8.8.8.8 deu 100% packet loss na nossa VM?

As 2 Rotas Automáticas de toda VPC

- 1. Rota Local (10.<ID>.0.0/24 -> Virtual Network): entrega pacotes dentro da própria sub-rede.
- 2. Rota Padrão (0.0.0.0/0 -> Default Internet Gateway): envia pacotes externos para o gateway de internet da VPC.

Se a rota 0.0.0.0/0 existe, por que falhou?

- Endereços Privados (RFC 1918) não navegam na internet pública!
- Nossa VM possui apenas o IP privado 10.<ID>.0.2.
- O servidor 8.8.8.8 não saberia para qual rede 10.x.x.x devolver a resposta.
- O Internet Gateway descarta pacotes sem IP público ou sem Cloud NAT!

Como o SNAT (Source NAT) traduz IPs privados na saída

Ida: Tradução do Endereço de Origem

- 1. A VM cria o pacote: Origem 10.7.0.2:49152 -> Destino 8.8.8.8.
- 2. O Cloud NAT intercepta na saída e troca a origem por um IP público real (ex: 34.122.10.55:61001).
- 3. O Cloud NAT anota essa correspondência na tabela de estado.

Volta: Retorno Seguro e Unidirecional

- 1. O 8.8.8.8 responde para o IP público 34.122.10.55:61001.
- 2. O Cloud NAT consulta sua tabela e devolve o pacote para 10.7.0.2.
- Segurança máxima: ninguém na internet consegue iniciar uma conexão de fora para dentro através do Cloud NAT!

Dupla Dinâmica no Terraform: Cloud Router + Cloud NAT

1. Cloud Router (google_compute_router)

- É o Plano de Controle Regional da rede.
- Em cenários de VPN/Interconnect, gerencia sessões BGP.
- Para o Cloud NAT, funciona como o controlador regional onde vinculamos o serviço de tradução.

2. Cloud NAT (google_compute_router_nat)

- Serviço de tradução definido por software (sem VM de gargalo).
- nat_ip_allocate_option = "AUTO_ONLY": o GCP aloca IPs públicos automaticamente.
- ALL_SUBNETWORKS_ALL_IP_RANGES: atende toda a sub-rede regional.

Destravando a Internet da VM Privada com Cloud NAT

Roteiro (Checkpoint: labs/03-roteamento-nat/)

- 1. Criar nat.tf declarando google_compute_router e google_compute_router_nat.
- 2. Executar terraform plan e terraform apply.
- 3. Voltar ao SSH da VM privada e testar: ping -c 3 8.8.8.8 e curl ifconfig.me!

terraform/live-coding/nat.tf

```
resource "google_compute_router" "router_aluno" {
  name    = "router-${var.region}-${local.student_suffix}"
  network = google_compute_network.vpc_aluno.id
  region  = var.region
}

resource "google_compute_router_nat" "nat_aluno" {
  name   = "nat-${var.region}-${local.student_suffix}"
  router = google_compute_router.router_aluno.name
  region = google_compute_router.router_aluno.region
  nat_ip_allocate_option = "AUTO_ONLY"
  source_subnetwork_ip_ranges_to_nat =
"ALL_SUBNETWORKS_ALL_IP_RANGES"
}
```

Como conectar duas VPCs no Google Cloud? (Comparativo)

VPC Network Peering & Shared VPC

- VPC Network Peering (Nosso Lab 04): troca direta de rotas na SDN entre 2 VPCs (mesmo em projetos diferentes), sem latência extra.
- Shared VPC: um projeto Host centraliza a VPC e compartilha sub-redes com projetos de serviço da mesma Organização.

Cloud VPN & Private Service Connect

- Cloud VPN (HA VPN): túnel criptografado IPsec + BGP via internet; ideal para conectar Datacenter On-Premises ou outra Cloud.
- Private Service Connect (PSC): expõe apenas 1 serviço específico via endpoint privado, mesmo com conflito de CIDR.

As 3 Leis Inquebráveis do VPC Network Peering

1. Zero Overlap & 2. Acordo Bilateral

- Lei 1 — Proibido CIDR Overlap: duas redes em Peering jamais podem ter sub-redes com a mesma faixa de IPs (por isso usamos 10.<ID>.0.0/24!).
- Lei 2 — Acordo Bilateral: o Peering só muda de INACTIVE para ACTIVE quando o Aluno aponta para o Hub e o Hub aponta para o Aluno.

3. Não-Transitividade (Isolamento entre Alunos)

- Lei 3 — Peering NÃO é transitivo ("o amigo do meu amigo não é meu amigo").
- O Aluno 01 conversa com o Hub e o Aluno 02 conversa com o Hub.
- Porém o Aluno 01 NÃO enxerga a rede do Aluno 02! Seus ambientes continuam isolados.

Arquitetura do Hub Central: Suportando 40 Alunos

Limite do GCP: 25 Peerings por VPC

- O GCP possui limite padrão de 25 peerings ativos por VPC.
- Para suportar até 40 alunos na sala, aplicamos Sharding no Hub:
- • Alunos 01 a 20 -> vpc-hub-turma-a (10.0.1.0/24 | Alvo: 10.0.1.10).
- • Alunos 21 a 40 -> vpc-hub-turma-b (10.0.2.0/24 | Alvo: 10.0.2.10).

Auto-Aceitação via API no Cloud Run

- O serviço Cloud Run que exibe o Mural no telão possui permissão compute.networks.addPeering no Projeto Hub.
- Assim que o aluno registra seu Peering, o Hub cria o lado recíproco automaticamente via API, deixando o status ACTIVE na hora!

Construindo o VPC Peering com o Projeto Central

Passo a Passo (Checkpoint: labs/04-vpc-peering-hub/)

- 1. Adicionar hub_project_id no variables.tf.
- 2. Criar peering.tf conectando sua VPC à VPC do Hub correspondente ao seu student_id.
- 3. Rodar terraform apply e ver a rota 10.0.X.0/24 surgir na sua tabela de rotas!

terraform/live-coding/peering.tf

```
resource "google_compute_network_peering" "aluno_para_hub" {
  name         = "peering-${local.student_suffix}-to-hub"
  network      = google_compute_network.vpc_aluno.self_link
  peer_network = "projects/${var.hub_project_id}/global/networks/
${local.hub_vpc_name}"

  export_custom_routes = false
  import_custom_routes = false
}
```

Hora da Verdade: Acendendo seu Card no Mural Central!

Validação Ponta a Ponta da Arquitetura

- 1. Acesso à VM privada (10.<ID>.0.2) via túnel Cloud IAP.
- 2. Pacotes utilitários instalados via saída pelo Cloud NAT.
- 3. Pacote ICMP (ping) e HTTP POST viajando por IP privado pelo VPC Peering até a VM coletora do Hub e atualizando o Cloud Run!

Dentro do SSH da sua VM Privada

```
# Alunos 01 a 20 -> HUB_IP="10.0.1.10"
# Alunos 21 a 40 -> HUB_IP="10.0.2.10"
HUB_IP="10.0.1.10"

# 1. Teste a latência privada via VPC Peering:
ping -c 3 $HUB_IP

# 2. Envie seu Check-in para aparecer no Projetor!
curl -X POST http://$HUB_IP:8080/checkin \
  -H "Content-Type: application/json" \
  -d '{"student_id": 7, "student_name": "Seu Nome"}'
```

Como organizamos Terraform na vida real? (Modules)

O Problema da Duplicação em Produção

- Em empresas reais temos múltiplos ambientes (dev, staging, prod) e dezenas de projetos.
- Copiar e colar arquivos .tf soltos gera código impossível de dar manutenção e padronizar.

O Conceito de Módulos (modules/)

- Funciona como uma função reutilizável de engenharia:
- • variables.tf: contrato de entrada tipado e validado.
- • main.tf: recursos encapsulados com boas práticas.
- • outputs.tf: valores exportados para conectar outros módulos.

Os 3 Módulos Padrão do Nosso Repositório (terraform/modules/)

1. vpc-network & 2. firewall-rules

- modules/vpc-network: encapsula VPC Custom, Sub-rede regional, Cloud Router e Cloud NAT (ativado via enable_nat = true).
- modules/firewall-rules: provisiona regras padronizadas de SSH via Cloud IAP (35.235.240.0/20) e ICMP interno.

3. vpc-peering & Reaproveitamento

- modules/vpc-peering: padroniza o fechamento de Peering entre Spoke e Hub.
- Todos os 3 módulos já estão 100% construídos e documentados no repositório para você usar após o curso!

Como toda a nossa rede vira 3 chamadas de Módulo limpas

Vantagens da Arquitetura Modular

- Legibilidade imediata: em poucas linhas entendemos toda a topologia.
- Encadeamento seguro: firewall_rules consome module.vpc_network.network_name.
- Pronto para replicar em dev, homologação e produção!

terraform/labs/05-boas-praticas-modules/main.tf

```
module "vpc_network" {
  source      = "../../modules/vpc-network"
  project_id  = var.project_id
  region      = var.region
  vpc_name    = local.vpc_name
  subnet_name = local.subnet_name
  subnet_cidr = local.subnet_cidr
  enable_nat  = true
}

module "firewall_rules" {
  source       = "../../modules/firewall-rules"
  project_id   = var.project_id
  network_name = module.vpc_network.network_name
  name_suffix  = local.student_suffix
}
```

O que muda quando levamos esse Terraform para Produção?

Remote State no GCS & State Locking

- Em equipe, o terraform.tfstate nunca fica salvo na máquina local.
- Usamos backend "gcs" em um Bucket do Cloud Storage com versionamento e State Locking (impede dois apply simultâneos).

Versionamento Git & Pipelines CI/CD

- Módulos versionados com tags semânticas (?ref=v1.2.0).
- Pipelines no GitHub Actions / GitLab CI rodando terraform fmt, validate, tfsec e comentando o plan no Pull Request!

Desmontando a Infraestrutura com terraform destroy

O Grafo Reverso de Destruição

- Tentar deletar uma VPC manualmente no painel falha se houver sub-redes, VMs, routers ou peerings ativos.
- Com terraform destroy, o Terraform percorre o grafo de trás para frente e remove 100% dos recursos sem deixar sobras na fatura!

Terminal — Limpeza Completa (17h30)

```
# 1. Saia do SSH da VM privada
exit

# 2. Destrua os recursos na ordem inversa automática:
terraform destroy

# [-] google_compute_instance.vm_aluno
# [-] google_compute_network_peering.aluno_para_hub
# [-] google_compute_router_nat.nat_aluno
# [-] google_compute_router.router_aluno
# [-] google_compute_subnetwork.subnet_principal
# [-] google_compute_network.vpc_aluno
```

Parabéns! Olha o que você construiu em Código hoje

Checklist de Conquistas do Dia

- VPC Customizada + Sub-rede CIDR /24 planejada sem colisão.
- Firewall Stateful, Network Tags e SSH via Cloud IAP em VM privada.
- Saída segura para a internet com Cloud Router e Cloud NAT.
- Interconexão Hub-and-Spoke via VPC Network Peering.
- Organização profissional com Módulos Reutilizáveis do Terraform!

Obrigado, turma do e-info / FACAPE!

- Todo o código, os gabaritos das 5 etapas e os módulos ficam disponíveis no repositório para seus estudos e portfólio.
- Facilitador: Luan Sousa — Cloud Engineer @ SantoDigital | Ciência da Computação FACAPE.